Schedule-Aware Hybrid Recommendation for sweet.tv: Re-Ranking Popularity, Association Rules, ALS, CBF and Personalized PageRank

Student: Yulia Filatkina Team: Individual (group AVITO)
Email: yuafilatkina@edu.hse.ru Date: 2026-10-25
Assignment: Final Exam Quiz — TV Program Recommendation on sweet.tv (Kaggle `sweettv-tv-program-recommender`)
Code & final submission file: https://github.com/yulkoo/sweettv-exam-recsys

***Abstract*—We predict, for 1,260 sweet.tv users, the five TV programmes they will watch most often in weeks 31–42, from a view log (weeks 11–30) that carries no programme ids and must be joined to a broadcast schedule in which only ~11% of rows are labelled. We join views to airings by channel and interval overlap, reproduce the organisers' published target rule on a temporal hold-out, implement content-based filtering, association rules, implicit ALS and personalized PageRank, and combine them with a popularity baseline through candidate generation and LightGBM LambdaRank re-ranking that also sees what will air in each user's habitual channel and hour slots. On the hold-out (weeks 25–30) the hybrid reaches MAP@5 = 0.3249 against 0.2162 for popularity and 0.3142 for the best single method (PPR). Most of the signal is the user's own repeated viewing; the hybrid's extra gain over PPR is small but statistically clear.**

***Index Terms*—**implicit feedback, association rules, matrix factorization, personalized PageRank, learning to rank.

I. Introduction

Problem. The competition supplies a view log (`user_id, vsetv_id, start_time, stop_time, duraton`), three schedule files and a template with five programme ids per test user; the metric is MAP@5. A view records only *which channel* was on, so user–programme interactions must first be built by a join, and every model must extrapolate 12 weeks forward.

Motivation. On linear TV a programme is watchable only when it airs, on a channel the user tends to switch to. The competition's Data page states how the answer was built: per user and airing, screen time = watched time / airing duration; airings with screen time ≥ 0.8 are counted per programme; the 5 most frequent programmes form the answer [7]. Frequently aired programmes that the user habitually finishes therefore dominate the target.

Example. Test user `8377619604347126107` → final output `2400480 240081 2400508 20088 12002955`.

Contributions. (1) A join, target reconstruction and validation protocol, with every course method evaluated individually against a popularity baseline on one hold-out; (2) a schedule-aware re-ranking hybrid that is the best method in every activity tercile, checked for leakage, determinism and label-shuffle collapse.

II. Related Work

Implicit ALS with confidence weights follows Hu et al. [1]; association rules (support, confidence, lift) follow Agrawal et al. [2]; content-based profiles from item attributes follow Pazzani and Billsus [3]; personalized PageRank as a random walk with restart to a preference vector follows Haveliwala [4] and, for item graphs at scale, Pixie [5]; the re-ranker is LightGBM [6] with its LambdaRank objective.

Alternatives considered. User-/item-based kNN CF (lecture 3): subsumed by the co-view graph that PPR walks on. Two-tower retrieval (lecture 5): there are no user side features and only 4,895 users, so ALS already covers the latent-factor family. Hand-tuned weighted score/rank fusion: its weights would have to be tuned on the hold-out; the re-ranker learns them on earlier folds instead.

III. Method

A. Join, screen time and unlabelled airtime

A view is matched to every airing on the same channel (`vsetv_id = channel_id`) whose interval `[start_time, start_time+duration)` overlaps the view interval; fragments of the same user–airing are summed and capped at the airing length, giving **screen time** = watched seconds / airing duration — the organisers' own normalisation [7]. Shifts of −3…+3 h between log and schedule were tested; 0 h maximised the share of views starting within 2 min of an airing start, so no shift is applied. 92.8% of views (300k sample) overlap some airing. **Unlabelled airings (`tv_show_id = 0`) are dropped as items**, as the organisers instruct [7]; they hold 43.6% of overlapped seconds. Labelled overlaps give 3,014,129 user–airing pairs.

Two thresholds, two roles. (i) *Target*: an airing counts if screen time ≥ 0.8, and the answer is a user's 5 programmes with most such airings (ties: more watched seconds, then smaller id — the organisers' tie rule is not published). (ii) *History interactions* fed to the methods: overlap ≥ min(300 s, 50% of the airing), weighted by screen time. The looser history cut keeps evidence from partially watched long airings: airings are long (median 5,700 s) while overlaps are short (median 712 s), and a 0.8 cut would discard most of the history. Weighting by screen time (rather than hours) was chosen on val (Table II).

B. Protocol shared by all methods

- **Split** (temporal, by airing start week): *val* = history wk 11–24 → target wk 25–30 (the hold-out for every number in Table I); *fit* = 11–18 → 19–24 and *fit2* = 11–15 → 16–21 (used only to fit the re-ranker); *test* = 11–30 → Kaggle wk 31–42. 1,233 of 1,260 users have a non-empty val target (1,104 have all five).
- **Candidate pool**: labelled programmes airing in the target window according to the schedule (5,013 in val). Re-recommending watched programmes is **allowed** for every method — the target is defined by repeated viewing.
- **Cold start**: a user with no watched labelled airing in history (8 in val, 2 in test) gets the popularity Top-5 in **every** method; ties and lists shorter than five are filled in popularity order.
- **Five ids**: each method scores every (user, candidate); its Top-5 is the arg-sort, ties broken by popularity. Scores are never compared across methods; the hybrid uses them as features.
- **MAP@5** = mean over all 1,260 users of Σ P@k·rel(k) / min(|truth|, 5); users with empty truth count as 0.

C. Methods (weights = screen time, decayed with an 8-week half-life)

- **Popularity** (required baseline): distinct users with a ≥ 0.8 airing of the programme in the last 4 history weeks.
- **History** (helper, not counted): the user's own decayed screen-time sum per programme.

- **CBF**: TF-IDF item vectors over category, three genres, directors, first 10 actors, production decade and channel; user profile = weighted sum of watched items; cosine score.
- **Association rules**: transactions = user-week baskets; rules A→B with support ≥ 20 baskets, confidence ≥ 0.05, lift > 1; antecedents = the user's programmes in the last 4 history weeks; score = Σ confidence.
- **ALS** [1]: all users × history programmes, confidence log(1 + 10·w); 8 factors, λ = 0.05, 20 iterations (factors chosen on val from {8, 16, 32, 64, 128}; `analysis/als_grid.txt`).
- **PPR**: co-view graph, edge = cosine of log-weighted user vectors, top-50 neighbours, symmetrised, row-normalised; restart probability 0.3 to the user's weight distribution, 20 power iterations.
- **Hybrid**: candidates = union of each method's Top-100, the Top-100 programmes airing most often in the user's (channel, hour-of-week) slots and channels, and every history programme that airs again (~404 per user; they contain 83.3% of val truth items). Features: each method's score and within-user rank; per-week history rates (airings, ≥ 0.8 airings overall / last 4 / last 2 weeks, screen-time sum, recency); item popularity; target-window airings per week; *chan_nair* / *slot_nair* = the user's channel or (channel, hour) shares × airings per week of the programme there; and *exp80* = past share of the programme's airings the user finished × airings per week ahead. All schedule features are per week, because target windows are 6 weeks in validation and 12 in test. LightGBM LambdaRank (one query per user), graded label = number of ≥ 0.8 airings in the target window (capped at 30). Fitted on *fit*+*fit2* and scored on *val*; for Kaggle refitted on *fit2*+*fit*+*val* and applied to *test*.

D. Tools and configuration

Python 3.12.13; pandas 3.1.0, NumPy 2.5.4, SciPy 1.18.1, scikit-learn 1.9.1 (TF-IDF), implicit 0.7.3 (ALS), LightGBM 4.7.0 (500 trees, lr 0.03, 63 leaves, ≥ 100 rows/leaf, 0.8 row/column sampling, truncation 20). Seeds: 42 (ALS, LightGBM), 0 (bootstrap). Apple M2, 8 GB, macOS 26.6.2. Runtime: join 21.5 s; single methods 0.2–4.4 s each on val; full final script (four folds, all methods, two fits, submission) 137 s. Two runs produced byte-identical submissions (md5 `8e24c2b1…`).

Key design decisions. (i) The target-week schedule is used only for "which programme airs on which channel at which time" (§5 of the task); target-week views are never read (the log ends in week 30). (ii) Re-ranker labels come from earlier folds; val is used only for selection. (iii) The label is the frequency the official answer is ranked by, not a binary hit.

IV. Experiments

A. Results on the hold-out

Method (val: wk 11–24 → 25–30)	MAP@5	light	medium	heavy	seen
Popularity (baseline)	0.2162	0.1436	0.2078	0.2971	0.839
Personal history (helper)	0.3106	0.1925	0.3160	0.4235	0.991
Content-based filtering	0.1914	0.1366	0.2038	0.2339	0.647
Association rules	0.2005	0.1333	0.1978	0.2703	0.863
Matrix factorization (ALS)	0.1695	0.1543	0.2020	0.1523	0.896
Personalized PageRank	0.3142	0.1997	0.3179	0.4249	0.990
Hybrid: candidates + LambdaRank	**0.3249**	**0.2073**	**0.3253**	**0.4421**	**0.946**

Table I. MAP@5 on the hold-out, overall and by tercile of history watch hours (light ≤ 128.7 h, heavy ≥ 254.3 h); "seen" = share of Top-5 slots already watched in history.

Comparison vs baseline. Popularity scores 0.2162; the user's own repeated viewing is far stronger (history 0.3106, PPR 0.3142), while AR (0.2005), CBF (0.1914) and ALS (0.1695) fall below the baseline. The hybrid adds +0.1087 over popularity (paired bootstrap over users, 1,000 resamples: 95% CI [+0.0980, +0.1195]) and +0.0108 over PPR (CI [+0.0058, +0.0159]) and is the best method in all three terciles.

Choice (all scored on val)	MAP@5
History weight: watched hours	0.2235
History weight: # watched airings	0.2907
History weight: # airings with screen time >= 0.8	0.3016
History weight: sum of screen-time fractions (used)	**0.3106**
Re-ranker label binary (in official top-5), trained on fit	0.3186
Re-ranker label binary (in official top-5), trained on fit+fit2	0.3222
Re-ranker label # airings with screen time ≥ 0.8 (cap 30), trained on fit	0.3228
Re-ranker label # airings with screen time ≥ 0.8 (cap 30), trained on fit+fit2 (final)	**0.3249**
Re-ranker label log2(1 + # airings ≥ 0.8), trained on fit	0.3184
Re-ranker label log2(1 + # airings ≥ 0.8), trained on fit+fit2	0.3251
Re-ranker label top-5 graded by log2 count, trained on fit	0.3206
Re-ranker label top-5 graded by log2 count, trained on fit+fit2	0.3226

Table II. Selection on val: interaction weight for the history signal (8-wk half-life) and re-ranker label/training folds.

B. Kaggle result of the designated final submission

[TO FILL: screenshot of the Kaggle Submissions page showing the private (and public) score of `final_hybrid_lambdarank.csv`, plus the two numbers in text.]

C. Verification

- **Target reconstruction**: for 5 random users the val top-5 was recomputed with plain loops straight from the raw CSVs (no shared join code); all 5 sets equal the pipeline's (`analysis/check_truth_bruteforce.txt`).
- **Temporal separation**: in every fold the last history airing starts before the first target second; the log contains no airing in weeks 31–42 (`analysis/audit.txt`).
- **Label-shuffle control**: the re-ranker fitted on permuted labels scores 0.0218 on val — the gain is learned, not leaked.
- **Hand-checked AP@5**: user `8377619604347126107`, hits at ranks 1–4, |truth| = 5 → (1+1+1+1)/5 = 0.8 = `apk()`.
- **Cold start**: all 8 cold val users get exactly the popularity Top-5 in all seven rows of Table I.
- **Submission file**: 1,260 rows, template header and user order, five distinct non-zero ids per row; re-running gives the same md5.

V. Recommendation Examples and Strategy Analysis

Two users were chosen deterministically on the val fold: a **heavy viewer** (95th percentile of history hours) and a **light viewer** (10th percentile), both with a non-empty val target, so each method's Top-5 can be compared with their actual official top-5 for weeks 25–30. Bold = in that top-5; "new" = not watched in history.

Heavy viewer `4644788228960559738` **573.2 h, 441 progr. in wk 11–24; official top-5 in wk 25–30**	**#1**	**#2**	**#3**	**#4**	**#5**	**AP@5**
History top-5 by hours	2400480 ТСН (Инфо)	12001682 М/с 'Губка Боб Квадратные Штан (Детям)	240081 Факты (Инфо)	2400508 Сегодня (Инфо)	2400467 Викна-Новости (Инфо)	
Popularity (baseline)	**2400480** **ТСН (Инфо)**	240081 Факты (Инфо)	2400467 Викна-Новости (Инфо)	2400508 Сегодня (Инфо)	20088 Завтрак с 1+1'. Информационно-(Развлечения)	0.20
Personal history (helper)	**12001682** **М/с 'Губка Боб Квадратные Штан (Детям)**	**2400480** **ТСН (Инфо)**	240081 Факты (Инфо)	**12002856** **М/с 'Элвиннн!!! и бурундуки (Детям)**	**12001734** **М/с 'Волшебные покровители (Детям)**	0.71
Content-based filtering	**12001682** **М/с 'Губка Боб Квадратные Штан (Детям)**	**12002355** **М/с 'Кунг-фу панда: Удивительн (Детям)**	12002753 М/с 'Санджей и Крейг (Детям) *new*	12002792 М/с 'Бешеные кролики (Детям) *new*	**12002856** **М/с 'Элвиннн!!! и бурундуки (Детям)**	0.52
Association rules	**2400480** **ТСН (Инфо)**	240081 Факты (Инфо)	6500479 Мир наизнанку (Познавательное)	2400467 Викна-Новости (Инфо)	2400508 Сегодня (Инфо)	0.20
Matrix factorization (ALS)	**12001682** **М/с 'Губка Боб Квадратные Штан (Детям)**	240081 Факты (Инфо)	**2400480** **ТСН (Инфо)**	700389 Кто сверху? (Развлечения)	6500479 Мир наизнанку (Познавательное)	0.33
Personalized PageRank	**12001682** **М/с 'Губка Боб Квадратные Штан (Детям)**	**2400480** **ТСН (Инфо)**	240081 Факты (Инфо)	**12002856** **М/с 'Элвиннн!!! и бурундуки (Детям)**	**12001734** **М/с 'Волшебные покровители (Детям)**	0.71
Hybrid: candidates + LambdaRank	**12001682** **М/с 'Губка Боб Квадратные Штан (Детям)**	**12002856** **М/с 'Элвиннн!!! и бурундуки (Детям)**	**12001734** **М/с 'Волшебные покровители (Детям)**	**2400480** **ТСН (Инфо)**	240081 Факты (Инфо)	0.80
Final hybrid, test wk 31–42	12001682 М/с 'Губка Боб Квадратные Штан (Детям)	2400480 ТСН (Инфо)	240081 Факты (Инфо)	12002856 М/с 'Элвиннн!!! и бурундуки (Детям)	12001734 М/с 'Волшебные покровители (Детям)	—

Light viewer `5527432006266821597` 60.8 h, 74 progr. in wk 11–24; official top-5 in wk 25–30	**#1**	**#2**	**#3**	**#4**	**#5**	**AP@5**
History top-5 by hours	2400508 Сегодня (Инфо)	5800509 Реальная мистика (Развлечения)	2400480 ТСН (Инфо)	400357 Холостяк', 10 сезон, 7 эп., 1 (Развлечения)	10003014 Т/с 'Отдел 44 (Сериалы)	
Popularity (baseline)	**2400480** **ТСН (Инфо)**	240081 Факты (Инфо)	**2400467** **Викна-Новости (Инфо)**	2400508 Сегодня (Инфо)	20088 Завтрак с 1+1'. Информационно- (Развлечения) *new*	0.33
Personal history (helper)	2400508 Сегодня (Инфо)	12002805 М/с 'Моланг', 1 сезон, 7 с (Детям)	**2400480** **ТСН (Инфо)**	240081 Факты (Инфо)	10002840 Т/с 'Пёс (Сериалы)	0.07
Content-based filtering	2400508 Сегодня (Инфо)	5900256 Утро с Украиной (Инфо)	5900319 Абзац! (Инфо) *new*	**2400467** **Викна-Новости (Инфо)**	5900276 Центральное телевидение' с Вад (Инфо) *new*	0.05
Association rules	**2400480** **ТСН (Инфо)**	240081 Факты (Инфо)	**2400467** **Викна-Новости (Инфо)**	2400508 Сегодня (Инфо)	20088 Завтрак с 1+1'. Информационно- (Развлечения) *new*	0.33
Matrix factorization (ALS)	12001682 М/с 'Губка Боб Квадратные Штан (Детям) *new*	**2400480** **ТСН (Инфо)**	2400508 Сегодня (Инфо)	240081 Факты (Инфо)	**2400467** **Викна-Новости (Инфо)**	0.18
Personalized PageRank	2400508 Сегодня (Инфо)	**2400480** **ТСН (Инфо)**	12002805 М/с 'Моланг', 1 сезон, 7 с (Детям)	240081 Факты (Инфо)	**2400467** **Викна-Новости (Инфо)**	0.18
Hybrid: candidates + LambdaRank	2400508 Сегодня (Инфо)	5900256 Утро с Украиной (Инфо)	12001682 М/с 'Губка Боб Квадратные Штан (Детям) *new*	240081 Факты (Инфо)	**2400480** **ТСН (Инфо)**	0.04
Final hybrid, test wk 31–42	2400508 Сегодня (Инфо)	2400480 ТСН (Инфо)	2400467 Викна-Новости (Инфо)	6700482 Секретные материалы (Другое)	1100411 Вещдок (Познавательное)	—

TABLE IV. EACH METHOD'S TOP-5 FOR THE TWO USERS ON THE HOLD-OUT (ID, MOST FREQUENT TITLE, CATEGORY); BOLD = WATCHED IN WK 25–30.

Familiar vs new. Almost every list is familiar: 94.6% of the hybrid's val slots were already watched by the user, and only 15 of its 2,345 hits were programmes new to the user (`analysis/segments.txt`). CBF proposes the most new programmes (35% of slots) and converts 9 of them into hits; its unfamiliar picks for the heavy viewer (Санджей и Крейг, Бешеные кролики) were misses. On this data a "discovery" is mostly a miss, and the log cannot tell whether an unwatched programme would have been enjoyed.

How the methods differ. *Association rules* capture co-viewing within a user-week, but Σconfidence favours consequents with the most support, so AR collapses to 27 distinct ids in val (14 in test), almost all news, and gives both example users nearly the popularity list. *ALS* smooths across users: for the light viewer it inserts a cartoon the user never watched (Губка Боб) because the latent factor places it near news viewers; latent factors miss the individual repetition that defines this target. *CBF* helps where co-view signal is thin but taste is coherent — for the heavy viewer it stays in the kids' cartoon cluster (AP 0.52). *PPR* restarts on the user's own history, so its lists are habitual programmes re-weighted by graph neighbours (99% seen); it equals history for the heavy viewer (0.71). *Popularity* already does well for mainstream news viewers but cannot see that the heavy viewer mostly finishes cartoons (0.20).

Why the hybrid. It is best on val overall (0.3249) and in every tercile (light 0.2073 vs 0.1997 PPR and 0.1436 popularity). Its top features by gain are PPR rank (0.477) and PPR score (0.228), followed by schedule context (*slot_nair* 0.035, *chan_nair* rank 0.029): it trusts the personalised graph score and adjusts it by how often a programme will air in the user's slots. For the heavy viewer it ranks the three cartoons the user finishes most above the news (AP 0.80). Cost is modest — 137 s end-to-end on a laptop, each single model < 5 s — so daily retraining is feasible. For the user the lists are dependable but conservative: 96.3% of test slots were already watched, 52% are news and 21% kids' programmes.

Failure case and limitations. The light viewer shows the failure mode: hybrid AP 0.04 vs 0.33 for popularity. The user's most-watched programme by hours (Сегодня) and Утро с Украиной did not make the user's ≥ 80% top-5, a cartoon the user never watched (Губка Боб, also ALS's top pick) entered the list, ТСН came only 5th and Викна-Новости was missed. More generally: only 10.8% of schedule rows carry an id, so 43.6% of viewing time is invisible to every method; the organisers' tie rule is unknown; validation covers 6 weeks while the test target counts 12; absence from the log does not prove a user never saw a programme elsewhere. No online experiment was run and no business impact is claimed.

VI. DISCUSSION

Failure case (wrong target). Our first protocol scored a programme as relevant if it was watched ≥ min(300 s, 50%) at least once in the target window; under it every method looked strong (popularity 0.6544, hybrid 0.7410), far above the best original leaderboard score (0.42912). **Root cause:** the official answer is the 5 programmes with most airings watched to ≥ 80% [7] — a much stricter, frequency-ranked target. **Fix + verification:** we re-implemented that rule (checked against a brute-force recomputation), switched the method weights to screen time and the re-ranker label to the ≥ 0.8 frequency; popularity fell to 0.2162 and the hybrid to 0.3249, much closer to the original leaderboard's

range (its 6-week val target is noisier than the 12-week test target).

Other defects found and fixed. (1) pandas 3 parses these timestamps as `datetime64[us]`, so `astype('int64') // 10**9` gave units of 1,000 s; the schedule looked self-overlapping (mean gap −2,226 s) and only 48.4% of views matched. Casting to `datetime64[s]` gave a median gap of 0 s and 92.8% matched views. (2) Methods with fewer than five positive scores were padded with the lowest programme ids instead of the popularity fallback (238 users for AR). (3) Schedule features were window totals, i.e. twice as large in the 12-week test as in the 6-week folds the reranker was trained on; they are now per-week rates.

What worked. (i) Screen-time weights instead of hours (history 0.2235 → 0.3106); (ii) schedule-driven candidates, which raised candidate recall of val truth from 0.768 to 0.833 (session log; final value in `analysis/select_labels.txt`); (iii) training on the frequency label.

What surprised / did not work. (i) ALS gets worse with more factors (0.1695 at 8, 0.0939 at 64) and is weakest for heavy viewers; (ii) association rules collapse to 27 distinct ids, mostly news; (iii) the many schedule features add only +0.0108 over PPR: about a third of truth items are programmes the user never watched in history, and only 15 such hits reach the hybrid's Top-5.

Next improvement. Model new-season and new-series premieres explicitly (a programme id absent from the history schedule that starts in the user's habitual slot), the main source of the remaining misses.

VII. AI Usage Disclosure

AI tool: Claude Code (CLI) with Claude Opus 5.5 (`claude-opus-5-5`), one session; the unmodified session export is submitted alongside this PDF. **How AI was used:** code generation (join, methods, hybrid, audit scripts), debugging (timestamp unit, tie handling, per-week normalisation), finding the official target rule on the competition's Data page via the Kaggle API, reference lookup (Crossref check of every DOI), drafting this report from the saved outputs in `analysis/`. Selection used val only, but many variants were compared on it (weights, ALS factors, labels), so the val numbers are slightly optimistic. Kaggle submissions were uploaded by me through the website.

[TO FILL BY STUDENT — What I personally verified: describe in your own words the checks you did yourself, e.g. recomputing one AP@5 by hand from `analysis/examples.txt`, opening the submission CSV, checking the Kaggle score on the screenshot, opening each reference link.**]**

[TO FILL BY STUDENT — What I trusted without verification: e.g. library internals of implicit/LightGBM, PPR hyperparameters (not tuned).**]**

References

[1] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 263–272. [Online]. Available: https://doi.org/10.1109/ICDM.2008.22. [Accessed: 2026-10-24].

[2] R. Agrawal, T. Imieliński, and A. Swami, "Mining association rules between sets of items in large databases," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 1993, pp. 207–216. [Online]. Available: https://doi.org/10.1145/170035.170072. [Accessed: 2026-10-24].

[3] M. J. Pazzani and D. Billsus, "Content-based recommendation systems," in *The Adaptive Web*, LNCS vol. 4321, Springer, 2007, pp. 325–341. [Online]. Available: https://doi.org/10.1007/978-3-540-72079-9_10. [Accessed: 2026-10-24].

[4] T. H. Haveliwala, "Topic-sensitive PageRank," in *Proc. 11th Int. Conf. World Wide Web (WWW)*, 2002, pp. 517–526. [Online]. Available: https://doi.org/10.1145/511446.511513. [Accessed: 2026-10-24].

[5] C. Eksombatchai *et al.*, "Pixie: A system for recommending 3+ billion items to 200+ million users in real-time," in *Proc. World Wide Web Conf. (WWW)*, 2018, pp. 1775–1784. [Online]. Available: https://doi.org/10.1145/3178876.3186183. [Accessed: 2026-10-24].

[6] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://papers.nips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html. [Accessed: 2026-10-24].

[7] sweet.tv — TV program recommender, Kaggle community competition, "Data" and "Evaluation" pages. [Online]. Available: https://www.kaggle.com/competitions/sweettv-tv-program-recommender/data. [Accessed: 2026-10-24].